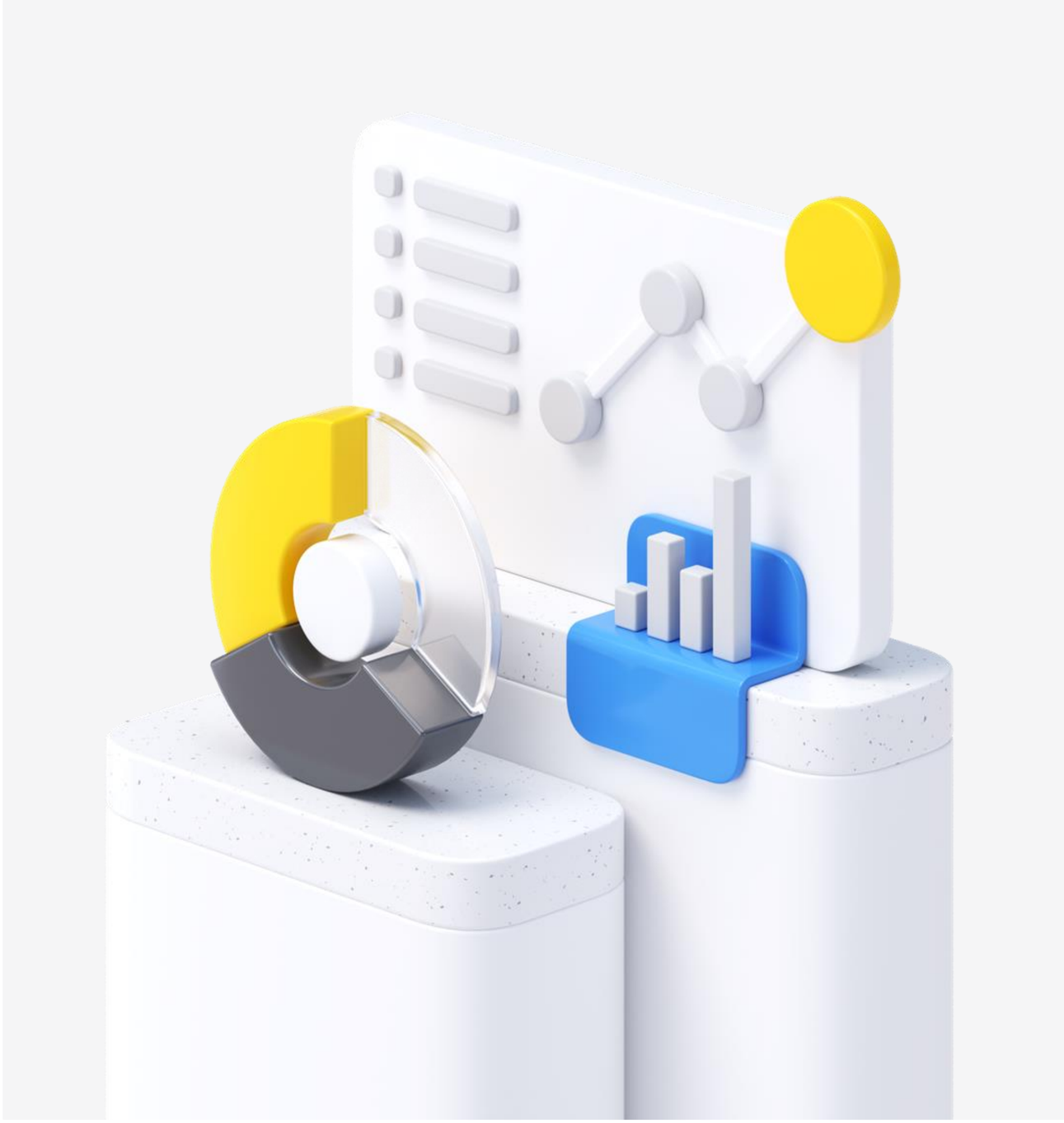
Лекция 1

Spring Framework. Основы



Чем фреймворк отличается от библиотеки?

Чем фреймворк отличается от библиотеки?

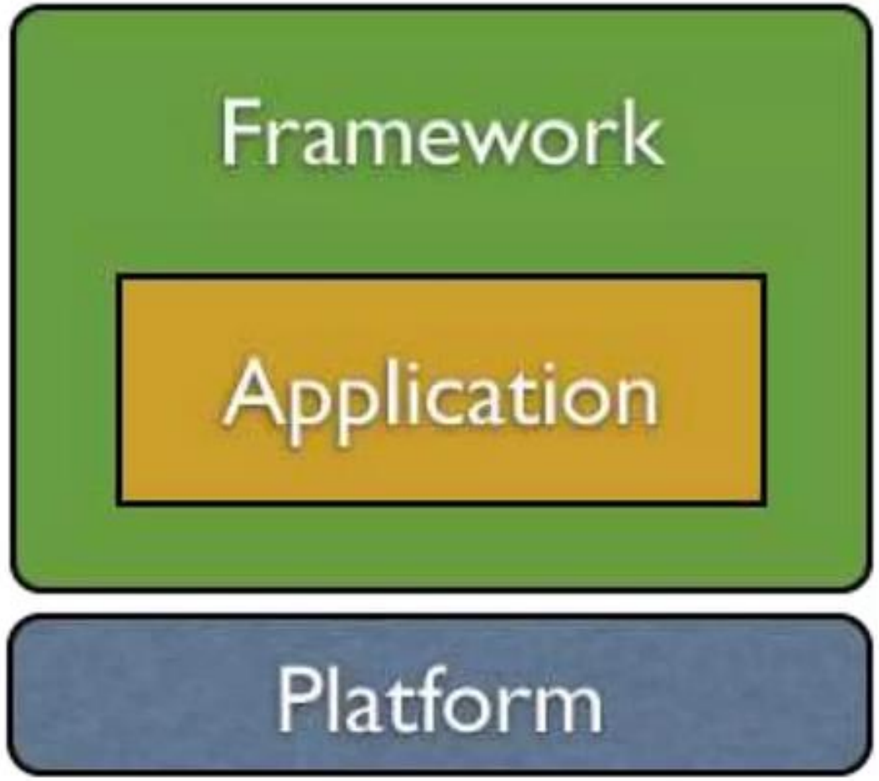
Библиотека — это набор готовых функций или классов, которые мы вызываем из своего кода. Программа управляет выполнением, а библиотека лишь помогает решить конкретную задачу.

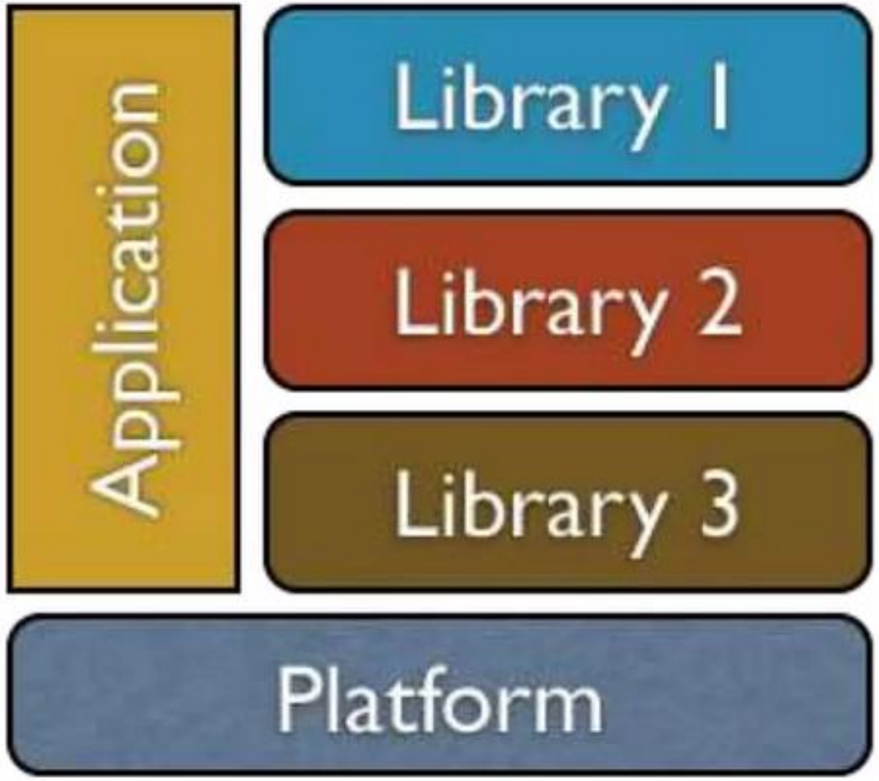
Пример: мы сами пишем приложение и при необходимости вызываем функции для работы с датой, сетью или JSON.

Фреймворк — это каркас приложения, который управляет основным потоком выполнения. Мы подключаемся к нему и вставляем свой код в заранее определённые точки расширения.
Главное отличие — принцип инверсии управления (Inversion of Control):
– при использовании библиотеки мы управляем кодом
– при использовании фреймворка фреймворк управляет нашим кодом

В библиотеке: “Мы вызываем код”.
Во фреймворке: “Код вызывает нас”.

Чем фреймворк отличается от библиотеки?





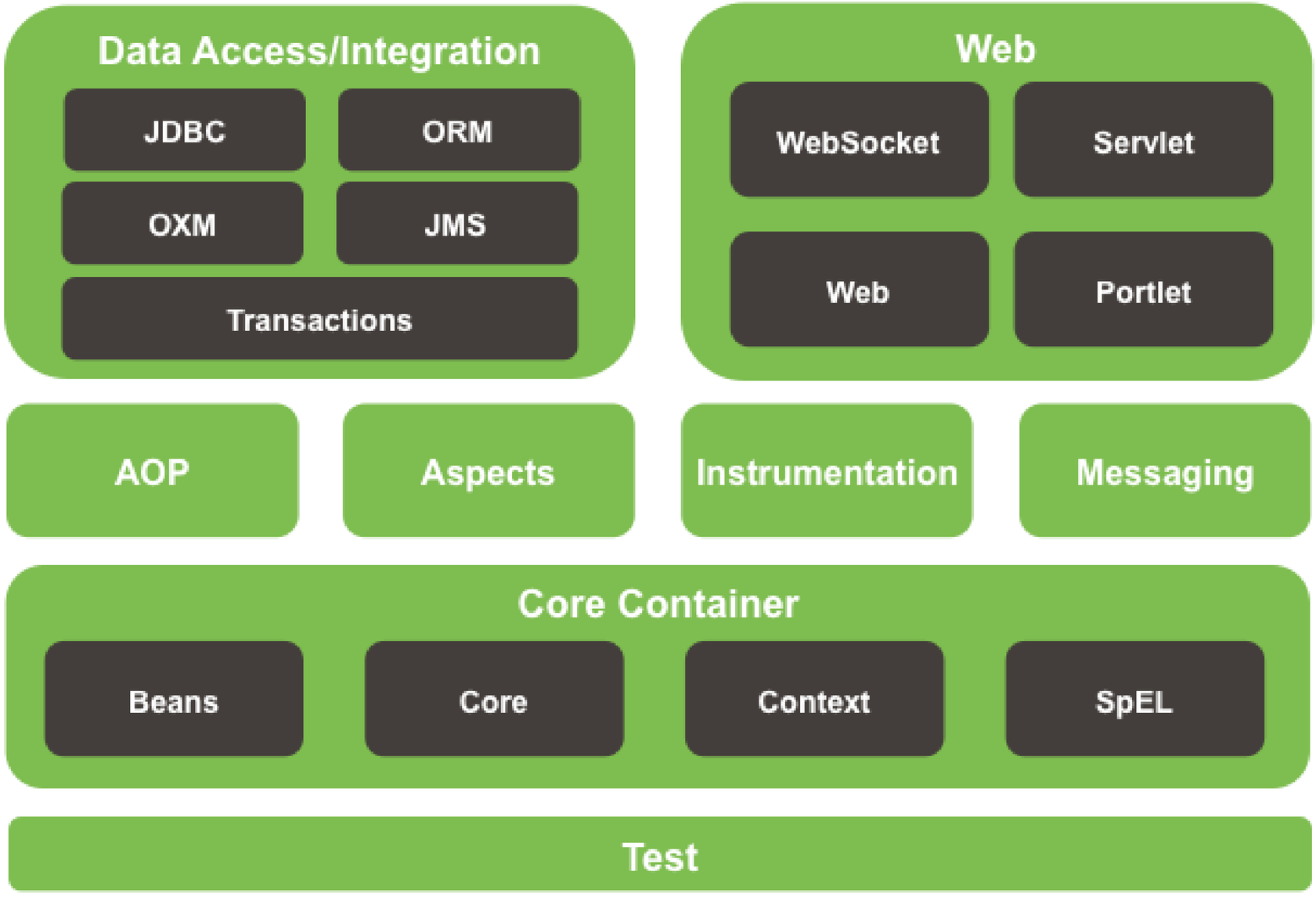
Что такое Spring Framework?

Spring Framework - мощный, модульный, enterprise-ориентированный фреймворк для Java (и Kotlin). Появился в 2003 году как альтернатива тяжёлым EJB (Enterprise JavaBeans)

Цель: упростить разработку сложных приложений за счёт:

- Автоматизации рутинных задач
- Управления зависимостями
- Интеграции с другими технологиями (БД, HTTP, Message Brokers и т.д.)

Spring Framework Runtime



Модуль	Назначение
Spring Core	База: IoC-контейнер, DI, утилиты
Spring Context	Расширение Core: доступ к бинам, профили, событийная модель
Spring AOP	Поддержка аспектно-ориентированного программирования
Spring Data Access	Работа с БД: JDBC, JPA, транзакции
Spring Web	Поддержка веб-приложений (MVC, REST)
Spring Security	Аутентификация и авторизация
Spring Messaging	Поддержка WebSocket, STOMP, JMS
Spring Integration	Интеграция с внешними системами
Spring Batch	Пакетная обработка данных
Spring Cloud	Микросервисы, конфигурация, сервис-дискавери

Рассмотрим Spring Core

Основные понятия

- **Container (ApplicationContext)** — главный компонент Spring, который управляет бинами: создает их, настраивает и связывает между собой.
- **Bean -** объект, который создается, настраивается и управляется контейнером Spring.
- **IoC (Inversion of Control) -** принцип, при котором управление жизненным циклом объектов передается от программиста контейнеру.
- **DI (Dependency Injection) -** паттерн реализации **IoC**, когда контейнер сам внедряет зависимости в объект при его создании.

IoC (Inversion of Control) и Container

IoC (Inversion of Control) - это принцип проектирования, при котором управление объектами передаётся **внешнему контейнеру**, а не создаётся внутри самого класса.

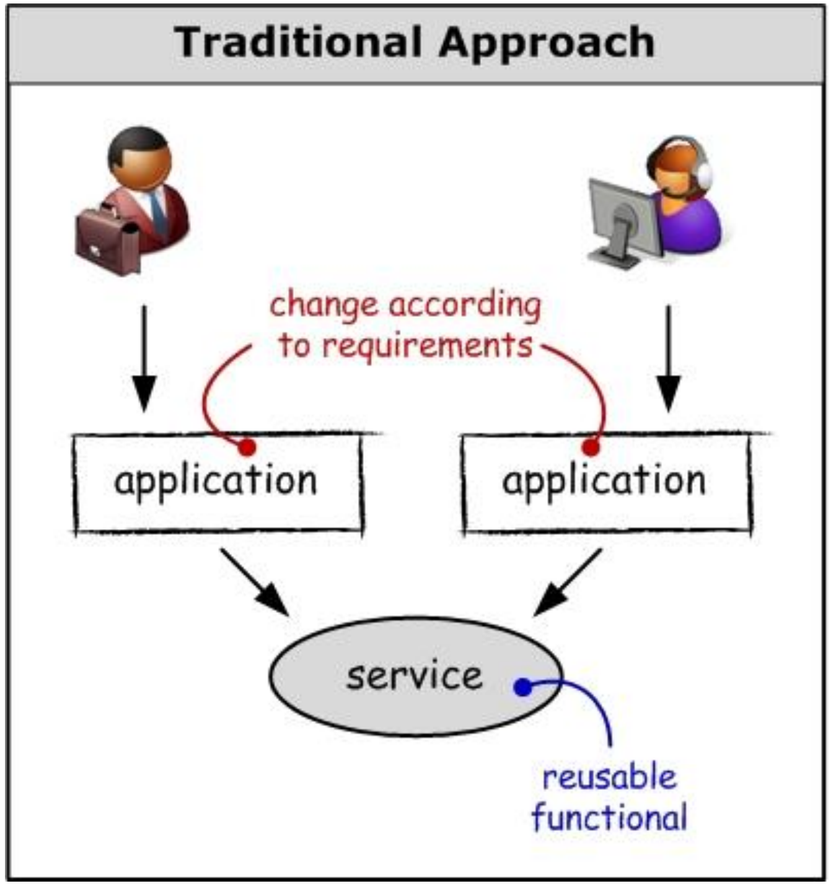
Как работает Контейнер:

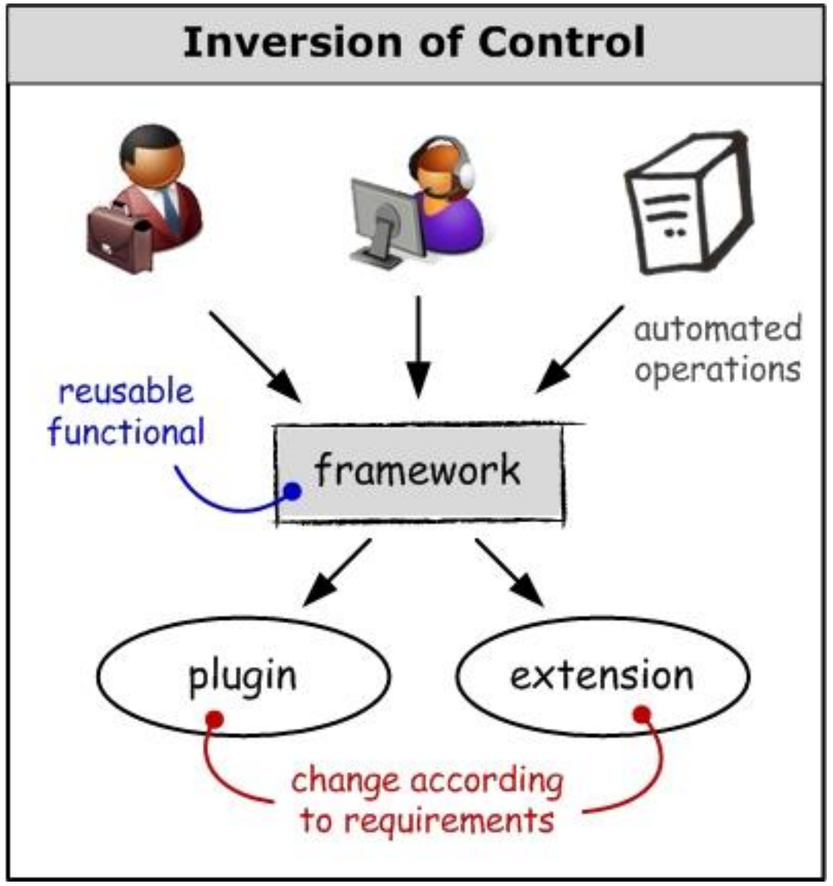
Сканирование: Spring ищет классы, помеченные аннотациями (@Component, @Service и др.), или методы с @Bean.

Регистрация Definition: Для каждого найденного класса создается BeanDefinition. Это «чертеж» будущего объекта: какой у него тип, область видимости (Scope), зависимости и методы инициализации.

Хранение: Все созданные объекты (синглтоны) хранятся в BeanMap внутри контейнера и раздаются по требованию.

IoC (Inversion of Control) и Container





DI (Dependency Injection)

DI (Dependency Injection) - паттерн реализации **IoC**, когда контейнер сам внедряет зависимости в объект при его создании.

Способы внедрения:

- **Через конструктор:** Гарантирует неизменяемость (final поля) и облегчает тестирование. Бин не будет создан, пока не будут найдены все зависимости.
- **Через сеттеры:** Позволяет внедрять зависимости опционально или менять их позже.
- **Через поля (@Autowired):** Самый простой, но считается плохой практикой, так как скрывает зависимости и усложняет unit-тесты.

Попробуем создать бин разными способами

Жизненный цикл бина

1. **Создание и инициализация**

- **Инстанцирование (Instantiating):** Контейнер находит определение бина (BeanDefinition) и создает экземпляр класса с помощью конструктора.
- **Установка свойств (Populate Properties):** Spring внедряет зависимости (DI) через @Autowired, сеттеры или поля.

1. **Пост-процессинг и настройка**

- **BeanPostProcessor (Pre-initialization):** Вызывается метод postProcessBeforeInitialization. На этом этапе работают прокси и аннотации, такие как @PostConstruct.
- **Инициализация (Initialization):**
  - Вызывается метод, помеченный @PostConstruct.
  - Если реализован интерфейс InitializingBean, вызывается метод afterPropertiesSet().
  - Вызывается кастомный init-method, если он указан в конфигурации.
- **BeanPostProcessor (Post-initialization):** Вызывается метод postProcessAfterInitialization. Именно здесь обычно создаются финальные AOP-прокси.

Жизненный цикл бина

3. Использование

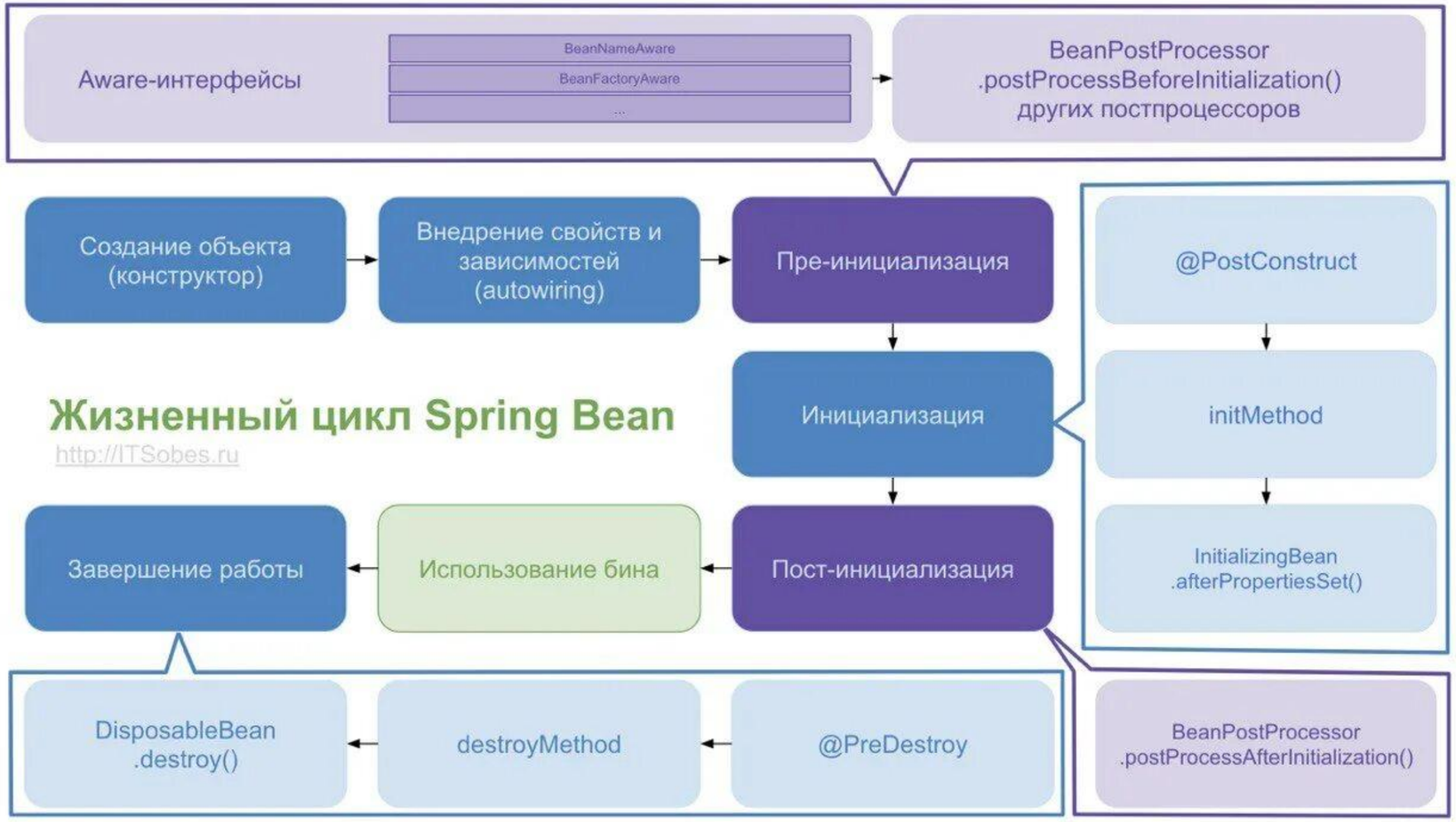
- **Ready to Use:** Бин полностью настроен и доступен приложению для работы.

4. Уничтожение (Destruction)

Когда контекст закрывается (например, при остановке приложения), происходят следующие шаги:

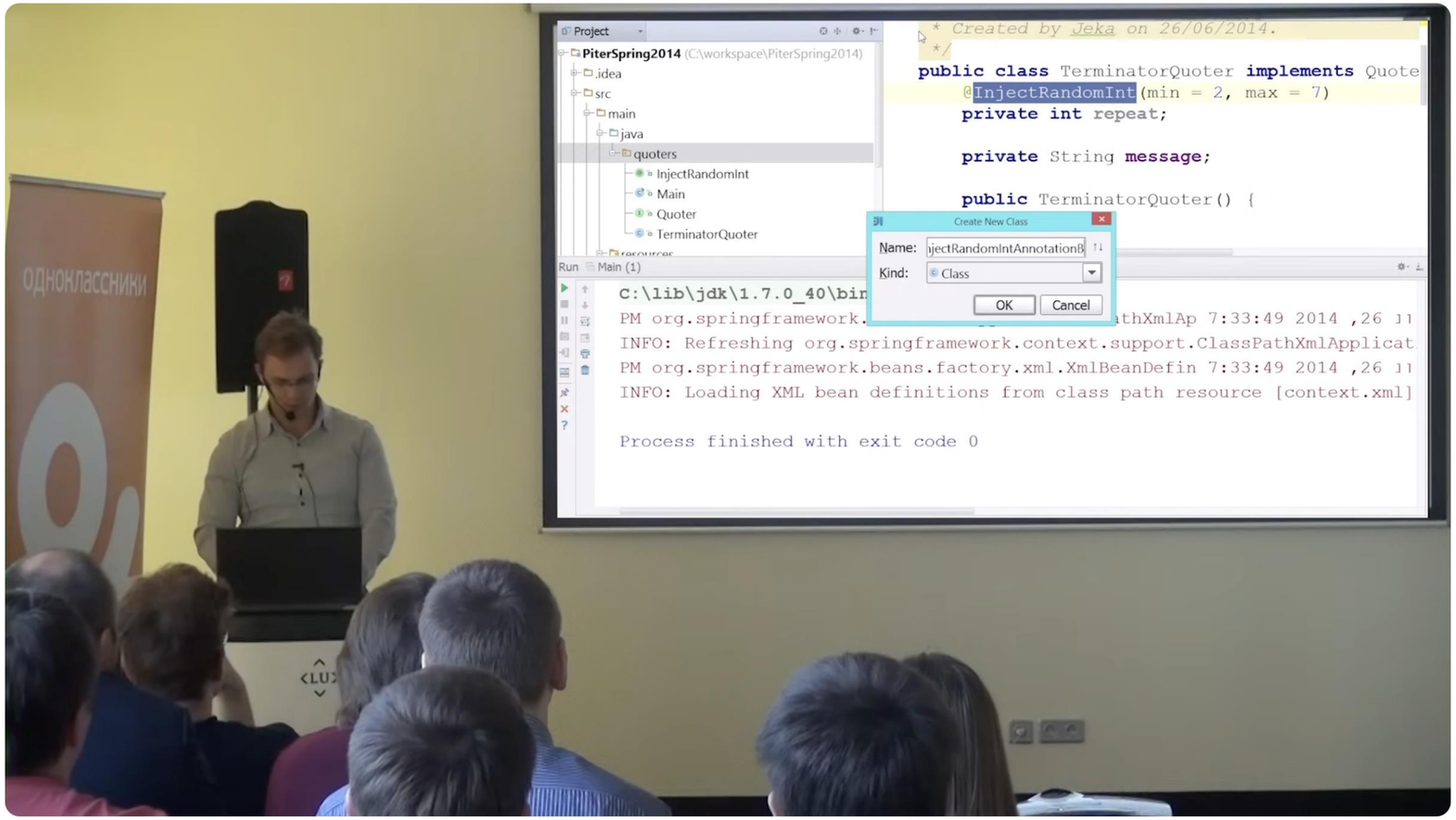
- **@PreDestroy:** Вызывается метод, помеченный этой аннотацией.
- **DisposableBean:** Если реализован этот интерфейс, вызывается метод destroy().
- **Custom destroy-method:** Вызывается метод уничтожения, определенный в конфигурации бина.

Жизненный цикл бина



Где узнать подробнее?

https://www.youtube.com/watch?v=BmBr5diz8WA



Евгений Борисов — Spring-потрошитель, часть 1

Что такое Spring Boot?

Spring Boot - это проект в экосистеме **Spring Framework**, предназначенный для упрощения процесса создания и запуска приложений на Spring. Он позволяет быстро развернуть готовое к работе приложение с минимальной настройкой и конфигурацией.

Особенности Spring Boot:

- ***Автоматическая конфигурация*** - Spring Boot автоматически настраивает компоненты приложения на основе подключённых зависимостей. Например, если в проекте есть `spring-webmvc`, он автоматически настроит Spring MVC.
- ***Упрощённое управление зависимостями*** - Использует стартеры (starters), как специальные зависимости, объединяющие часто используемые библиотеки. Например:
  - `spring-boot-starter-web` — для веб-приложений
  - `spring-boot-starter-data-jpa` — для работы с базами данных
  - `spring-boot-starter-security` — для безопасности

Примеры популярных стартеров:

Стартер	Что включает
`spring-boot-starter`	Core + Logging (Logback + SLF4J)
`spring-boot-starter-web`	Spring MVC, Jackson, Tomcat
`spring-boot-starter-data-jpa`	Hibernate, Spring Data JPA, транзакции
`spring-boot-starter-security`	Spring Security, защита endpoints
`spring-boot-starter-validation`	Bean Validation (JSR-380), Hibernate Validator
`spring-boot-starter-actuator`	`/health`, `/metrics`, `/info` и др.
`spring-boot-starter-test`	JUnit, Mockito, Spring Test, AssertJ

Структура простого Spring Boot проекта

Типичный проект Spring Boot на Kotlin создаётся с помощью Gradle (чаще всего) или Maven.
В Kotlin-проектах обычно используется Gradle Kotlin DSL (build.gradle.kts).

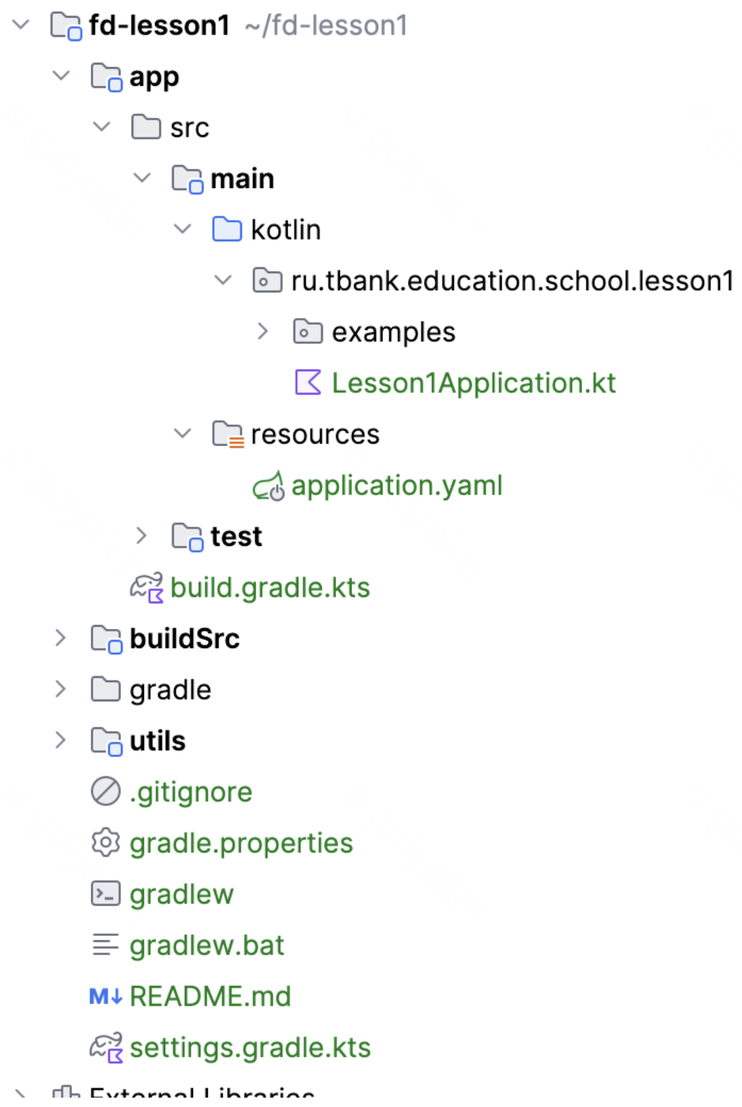
В корне проекта находятся:
– build.gradle.kts — файл зависимостей и настроек сборки
– settings.gradle.kts — имя проекта
– папка src
– папка build (создаётся автоматически после сборки)

Основная структура кода располагается в папке:
src/main/kotlin
Внутри неё:
– главный класс приложения с аннотацией @SpringBootApplication
– пакеты с логикой приложения (controller, service, repository и другие)

Структура простого Spring Boot проекта

Файл конфигурации располагается в:
src/main/resources

Там обычно находится:
– application.yml или application.properties — настройки приложения
(порт сервера, база данных, профили и другие параметры)



Базовый минимум Gradle

Конфигурация build.gradle.kts:

- **Plugins:** Подключение Spring Boot 3.5, Dependency Management и Kotlin-плагинов для Proxy-совместимости.
- **Toolchain:** Фиксация Java 17.
- **Dependencies:** Использование «стартеров» для минимизации кода и jackson-module-kotlin для корректной работы с Data-классами.
- **Compiler:** Настройка -Xjsr305=strict для интеграции Null-safety Kotlin со Spring.

```kotlin
plugins {
    kotlin("jvm") version "1.9.25"
    kotlin("plugin.spring") version "1.9.25"
    id("org.springframework.boot") version "3.5.9"
    id("io.spring.dependency-management") version "1.1.7"
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of( version = 17)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("org.jetbrains.kotlin:kotlin-test-junit5")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}

kotlin {
    compilerOptions {
        freeCompilerArgs.addAll( ...elements = "-Xjsr305=strict")
    }
}

tasks.withType<Test> {
    useJUnitPlatform()
}
```

Базовый минимум Gradle

Задача	Назначение
`./gradlew build`	Сборка JAR-файла (в `build/libs/`)
`./gradlew bootRun`	Запуск приложения "на лету" (как `java -jar`)
`./gradlew test`	Запуск unit- и integration-тестов
`./gradlew clean`	Удаление папки `build/`
`./gradlew dependencies`	Показывает дерево зависимостей
`./gradlew dependencyInsight --dependency <name>`	Анализ конфликта версий
`./gradlew jar`	Создание JAR (без Spring Boot упаковки)
`./gradlew bootJar`	Создание исполняемого JAR с зависимостями

Давайте рассмотрим примеры